

Project Documentation: Timișoara Real Estate AI Price Estimator

Zeno-Tudor Tomodan
zeno.tomodan05@e-uvt.ro

June 2, 2026

1 Introduction

The Timișoara Real Estate AI Price Estimator is an intelligent desktop application designed to assist users in property valuation. By combining a Java-based graphical interface with a local Large Language Model (LLM), the application provides accurate, data-backed property estimations tailored to the specific market dynamics of Timișoara and its metropolitan area (Dumbrăvița, Giroc, Moșnița, etc.).

2 System Architecture

The application operates on a client-server local architecture:

1. **Frontend:** A Java Swing GUI collects user input (Zone, Area, Rooms, etc.).
2. **Prompt Engineering:** Data is aggregated into a structured prompt and sent to the Ollama API via HTTP.
3. **AI Backend:** The `timisoara-ai` model (based on Qwen2.5:3b) processes the input, using a pre-defined dataset of 130+ listings as context.
4. **Response Parsing:** The raw string output is parsed to decouple the financial figure from the descriptive negotiation advice.

3 Core Implementation Logic

3.1 Input Capture and Prompt Generation

Data is captured via the `startPredictionTask()` method. This method extracts values from the UI components and formats them into a strict prompt structure to ensure the AI responds in the expected format.

```
1 private void startPredictionTask() {
2     String zone = (String) zoneCombo.getSelectedItem();
3     String area = areaField.getText();
4     String rooms = roomsField.getText();
5     // ... Additional fields extracted ...
6
7     String prompt = String.format("Analizeaza urmatorul apartament:\nZona: %s\
8         nSuprafata: %s\mp\n...",
9         zone, area, rooms);
10
11     String jsonPayload = String.format("{\"model\": \"%s\", \"prompt\": \"%s\", \"
12         stream\": false}",
13         MODEL_NAME, prompt);
14
15     // Execute worker...
```

```
13 }
```

Listing 1: Input Capture and Prompt Construction

3.2 Response Formatting

The AI returns a combined string of the price and negotiation advice. We use string parsing to separate these elements for a better user experience.

```
1 private void formatAndDisplayResponse(String fullResponse) {
2     if (fullResponse.contains("ESTIMARE_PRET:")) {
3         String[] parts = fullResponse.split("ARGUMENTE_NEGOCIERE:");
4         if (parts.length >= 2) {
5             String rawPriceLine = parts[0].replace("ESTIMARE_PRET:", "").trim();
6             finalPrice = rawPriceLine.substring(0, rawPriceLine.indexOf("(")).trim();
7             finalAnalysis = "ARGUMENTE_NEGOCIERE:\n" + parts[1].trim();
8         }
9     }
10    priceBoxLabel.setText(finalPrice);
11    resultArea.setText(finalAnalysis);
12 }
```

Listing 2: Parsing Logic to isolate the Price

4 User Interface

The graphical interface is designed for clarity. The user inputs property details, and upon clicking the CERE ANALIZA AI button, the application displays the calculated price in a prominent, high-visibility label, followed by the AI's analytical breakdown.

AI Real Estate Timisoara

Estimează Prețul

Zona / Cartier

Aradului

Suprafață utilă (m²)

72

Număr de camere

3

Anul construcției

2026

Tip Încălzire

Încălzire Centralizată

Facilități suplimentare

- ☐ Balcon
- ☐ Lift / Elevator
- ☐ Aer Condiționat (AC)

CERE ANALIZA AI

0 €

Aștept datele pentru a începe analiza...

Figure 1: The Main Application Interface

5 AI Model Architecture

The system utilizes a custom `Modelfile` that anchors the LLM to real-world data. Key components of the model include:

- **Dataset:** 130+ real estate listings provided as CSV, which the model uses for comparative analysis.
- **Inference Rules:** Explicit instructions on how to weight factors (e.g., penalizing the lack of an elevator in older city blocks vs. newer metropolitan constructions).
- **System Prompt:** Forces the AI into a rigid output structure, facilitating the parsing logic implemented in the Java layer.

6 Required Environment

To execute the application successfully, the following environment is required:

- **Java Development Kit (JDK):** Version 11 or higher.
- **Ollama Service:** Running locally on port 11434.
- **Model Creation:** The model must be registered using: `ollama create timisoara-ai -f Modelfile`